Advanced Data Structures

Lecture 4 – Amortized Analysis III: The Potential Method

Dr. Suraj Kumar

Department of Computer Science & Engineering

4 August 2026

*Amortized analysis, part 3 of 3 (aggregate → accounting → **potential**)*

Where We Are

Recall: the accounting method

Charge each operation $\hat{c}_i$; bank the surplus as credit stored on individual objects; spend it on expensive operations, keeping the total credit ≥ 0.

Its inconvenience

You must decide *where* each unit of credit sits and re-argue the invariant by hand for every new structure.

Three methods

1. Aggregate – Lecture 2
2. Accounting – Lecture 3
3. **Potential** – *today*

Today: replace scattered credit by a *single number* computed from the structure's state.

The Potential Method

Credit as a function of state

Definition (potential method)

A **potential function** Φ maps each state D of the data structure to a real number $\Phi(D)$. If D_i is the state after the i-th operation, the **amortized cost** is

$$\hat{c}_i \;=\; c_i \;+\; \Phi(D_i) - \Phi(D_{i-1}).$$

$$\sum_{i=1}^{n} \hat{c}_i = \sum_{i=1}^{n} c_i \;+\; \Phi(D_n) - \Phi(D_0).$$

So $\sum c_i = \sum \hat{c}_i - \big(\Phi(D_n) - \Phi(D_0)\big)$.

If $\Phi(D_i) \geq \Phi(D_0)$ for all i – most easily by choosing

$$\Phi(D_0) = 0 \quad \text{and} \quad \Phi(D) \geq 0,$$

then $\sum c_i \leq \sum \hat{c}_i$.

Think of Φ as a bank balance fixed by the current configuration: high potential = stored work ready to pay for a future expensive step.

Example 1 – Stack with MULTIPOP

Potential = number of items on the stack

Choose the potential

$$\Phi(D) = |S| \quad \text{(stack size)}.$$

$\Phi(D_0) = 0$ and $\Phi \geq 0$, so the condition holds automatically.

One line per operation

$$\begin{aligned} \text{PUSH}: &\quad \hat{c} = 1 + (+1) = 2, \\ \text{POP}: &\quad \hat{c} = 1 + (-1) = 0, \\ \text{MULTIPOP}(k'): &\quad \hat{c} = k' + (-k') = 0, \end{aligned}$$

where $k' = \min(k, |S|)$ items are removed.

$$\sum c_i \leq \sum \hat{c}_i \leq 2n \;=\; O(n), \qquad \hat{c} = O(1).$$

Exactly the accounting charges (2, 0, 0).

Example 2 – Binary Counter

Potential = number of 1-bits

Choose the potential

$$\Phi(D) = b \quad (\#\,1\text{-bits}).$$

The counter starts at 0, so $\Phi(D_0) = 0$ and $\Phi \geq 0$.

The bound

$$\sum c_i = \sum \hat{c}_i - (b_n - b_0) = 2n - b_n \leq 2n.$$

One Increment

Say it resets t bits $1 \to 0$ and sets one bit $0 \to 1$. Then

$$\begin{aligned} c_i &= t + 1, \\ \Delta\Phi &= (+1) - t, \\ \hat{c}_i &= c_i + \Delta\Phi = (t+1) + (1-t) = 2. \end{aligned}$$

The t cancels: costly increments lower the potential by exactly what they overspend.

Example 3 – Dynamic Table (Table Doubling)

A potential that tracks "how full" the table is

Choose the potential

$$\Phi(T) = 2 \cdot \text{num} - \text{size},$$

with num items in a table of capacity size.

Why $\Phi \geq 0$

Table doubling keeps the load factor between $\frac{1}{2}$ and 1 ($\text{num} \geq \text{size}/2$), so $\Phi \geq 0$. Also $\Phi(D_0) = 0$.

Endpoints

- Just after an expansion: $\text{num} = \text{size}/2 \Rightarrow \Phi = 0$.
- When the table is full: $\text{num} = \text{size} \Rightarrow \Phi = \text{size}$.

The potential climbs from 0 to size between expansions – exactly enough to pay for the next copy.

Example 3 – Amortized Cost Is 3 Either Way

Two cases, same answer

No expansion

Insert into spare room: $c_i = 1$, size unchanged, num up by 1:

$$\Delta\Phi = 2, \qquad \hat{c}_i = 1 + 2 = 3.$$

With expansion (was full, num = size = m)

Copy m items, then insert: $c_i = m + 1$; new capacity $2m$, so

$$\Delta\Phi = 2 - m, \qquad \hat{c}_i = (m+1) + (2-m) = 3.$$

$$\hat{c}_i = 3 \text{ in both cases} \implies \sum c_i \le 3n = O(n), \qquad \hat{c} = O(1).$$

Comparison across all three methods

Problem	Aggregate	Accounting	Potential
Multipop stack	$T(n) \leq 2n$	PUSH$= 2$, POP$= 0$	$\Phi = \lvert S \rvert$
Binary counter	$T(n) < 2n$	INCR$= 2$	$\Phi = \#\,1\text{s}$
Dynamic table	$T(n) < 3n$	INSERT$= 3$	$\Phi = 2 \cdot \text{num} - \text{size}$

How to choose

- **Aggregate** – when one total is easy to bound.
- **Accounting** – when a concrete "who pays" story is clearer.
- **Potential** – when the cost depends cleanly on state.

Why potential dominates later

For **splay trees** and **Fibonacci heaps** (Unit II), a well-chosen Φ is the *only* practical tool – accounting by hand becomes hopeless.

Summary

Key points

- Potential method: $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$.
- Telescoping: $\sum c_i = \sum \hat{c}_i - (\Phi_n - \Phi_0)$; need $\Phi_n \geq \Phi_0$ (take $\Phi \geq 0,\ \Phi_0 = 0$).
- **Multipop:** $\Phi = |S| \Rightarrow 2,\ 0,\ 0.$
- **Counter:** $\Phi = \#1\text{s} \Rightarrow 2$; total $2n - b_n$.
- **Table:** $\Phi = 2 \cdot \text{num} - \text{size} \Rightarrow 3.$
- Potential = accounting's total credit, written as a state formula.

Next lecture (L5)

Amortization in practice: applying the potential method to the augmented stack, binary counter, and dynamic table as a *design* tool – and a first look at Union–Find.

Reading

- CLRS, §17.3 (the potential method)
- Re-derive §17.1–17.2 examples with Φ

Exercises

Try before Lecture 5

1. Prove the telescoping identity $\sum_{i \le n} \hat{c}_i = \sum_{i \le n} c_i + \Phi(D_n) - \Phi(D_0)$, and deduce $\sum c_i \le \sum \hat{c}_i$ when $\Phi(D_n) \ge \Phi(D_0)$.
2. Using $\Phi = |S|$, extend Example 1 to $\textsc{Multipush}(S, x, k)$ (cost k). What amortized cost does the potential method assign it?
3. For the binary counter, confirm the exact total $2n - b_n$ against the eight-increment run from Lecture 2 or 3.
4. **(Insertions and deletions.)** Let the table also support $\textsc{Delete}$, contracting to half size when it becomes only $\frac{1}{4}$ full. Show that $\Phi = \left| 2\,\mathrm{num} - \mathrm{size} \right|$ (or the CLRS two-case potential) keeps both $\textsc{Insert}$ and $\textsc{Delete}$ at $O(1)$ amortized.
5. Show the potential method *subsumes* accounting: given any valid accounting scheme, define $\Phi =$ total banked credit and recover the same amortized costs.